# Sophix V3 Performance And Complexity Assessment

Date: 2026-06-01

Status: Architecture assessment after DD coherence review

Scope reviewed:

- 127 non-guide DD/frontend markdown documents.
- 116 developer implementation guides.
- Updated HLD and process/module map.
- Cross-module traceability, frontend mappings, and late gap-closure DDs.

## 1. Executive Verdict

The SOPHIX V3 design is coherent as a modular BSS design, but it is too heavy if interpreted as "one DD equals one service/pod/team/deployment".

The design is implementable for a medium-budget first deployment only if we apply these simplifications:

1. Treat DDs as capability contracts, not deployment boundaries.
2. Group low-volume modules into a small number of deployable runtimes.
3. Use Camunda only for real long-running workflows.
4. Use Drools only for configurable business policy.
5. Keep reporting, dashboards, and exports on REP-01.
6. Avoid all-to-all synchronous module communication.
7. Phase late capabilities instead of implementing every DD at full depth in the first release.

Without these controls, the design will become operationally expensive and slow to deliver. The risk is not only infrastructure cost; it is also debugging complexity, cross-team coordination, incident response, and API latency.

## 2. Main Assessment Answer

No, the modules should not all communicate with each other directly.

The intended model is:

- one module owns each command and authoritative state;
- a command calls only the few modules required to validate or execute that command;
- long-running work continues through Camunda/events;
- reporting and analytics use REP-01;
- cache is non-authoritative acceleration;
- dense UI screens use panelized reads or a BFF/read aggregator.

If implementation teams build every screen or process as live synchronous calls to many modules, the system will have avoidable latency and failure coupling.

## 3. Complexity Rating By Area

| Area | Complexity | Assessment | Simplification decision |
|---|---|---|---|
| Foundations | Medium | Auth, DB, Kafka, cache, Camunda, Drools, file storage are necessary. | Keep, but operate as shared platform, not per-module runtime. |
| Catalog/config | Medium | Many catalog DDs, mostly CRUD/reference. | Deploy together where practical; use cache; no Camunda unless approval lifecycle exists. |
| Customer/ILM | Medium | Central source of truth, widely used. | Keep separate logical owner. Use APIs/read models, not direct DB access. |

| Area | Complexity | Assessment | Simplification decision |
|---|---|---|---|
| Subscription SUB-LM/SUB-WF | High | Many workflows and state transitions. Complexity is justified because subscription state is critical. | Keep split between master state and operations. Consolidate workers. |
| Fulfillment/order | High | FUL-02 has many steps and dependencies. Complexity is justified but must be phased. | Implement minimum order-to-activation path first; defer rare branches. |
| Billing | High | Charging, payment, invoicing, wallet, dunning, tax, usage are broad. | Keep billing ownership strict. Run batch-heavy jobs off peak. Do not over-Camunda simple scanners. |
| WO/field execution | High | Field work, offline app, evidence, assignment, cancellation. Necessary complexity. | Keep WO separate. Keep field-app offline scope narrow. |
| OSR/equipment | High | Stock/equipment serials are accounting-grade state. Necessary separation from WO. | Keep OSR separate; phase procurement/audit after stock/instance basics. |
| Ticketing/ASR | Medium-high | Four ASR DDs can look like four modules. | Implement as TCK ticket types/workflow satellites, not separate services. |
| Commercial/campaign/discount/CVM | Medium-high | Important but can explode in scope. | MVP: rules-based campaigns/discounts; CVM as simple queues, not ML. |
| Reporting | Medium | REP-01 is needed to prevent dashboard fan-out. | Keep REP-01, but start with daily/near-real-time priority datasets only. |
| Frontends/channels | Medium-high | Many apps, but can share shell/components. | Backoffice/Admin/CRM/Reporting can be one web app with role navigation. Keep sales/contractor/self-care/USSD separate by UX/channel need. |

# 4. Areas To Simplify Now

## 4.1 Deployment Boundaries

Do not deploy 127 DDs as 127 services.

Recommended medium-budget deployment groups:

| Deployable group | Contains |
|---|---|
| API Gateway / Edge | Gateway routing, rate limits, auth verification, frontend API base. |
| Customer + Catalog API | ILM, PLM, SIP, RLM, tech regions, business config APIs. |
| Subscription API/Workflow | SUB-LM, SUB-WF operation APIs, subscription worker group. |
| Billing API/Jobs | Charging, payments, invoicing, wallet, dunning, tax, rating batch interfaces. |
| Fulfillment API/Workflow | FUL-02/03/04/05/07/08/09/10 and provisioning orchestration. |
| Field Operations API | WO, EM-02 contractor/staff, field audit process types. |
| Equipment/Stock API | OSR stock, equipment instance, RMA/swap, procurement, inventory audit. |
| CRM/Commercial API | TCK, ASR flow types, SALES, EM-01, campaigns, discount assignment, CVM. |
| Integration Workers | Payment gateway, provisioning, tax gateway, mediation/rating ingestion, external callbacks. |
| Reporting API | REP-01 data mart, dashboards, exports, reconciliation APIs. |
| Shared Workers | Consolidated low-volume worker pools with per-topic metrics. |

This keeps logical ownership while reducing operational overhead.

## 4.2 Camunda Usage

Keep Camunda for:

- FUL-02 order lifecycle.
- SUB-WF activation, termination, relocation, migration, non-payment suspension.
- WO install/shifting/support flows.

- OSR-RMA/swap flows.
- Approval processes with human tasks.
- Long external waits: payment wait, WO finalization wait, provisioning retry/compensation.

Do not use Camunda for:

- catalog CRUD;
- RBAC catalog CRUD;
- simple validation;
- daily report aggregation;
- basic invoice read APIs;
- simple cache invalidation;
- deterministic reconciliation scans that can be scheduled jobs.

## 4.3 Drools Usage

Keep Drools for:

- package/service eligibility;
- dunning policy;
- discount/campaign eligibility;
- approval requirement decisions;
- contractor/WO routing rules;
- field-audit discrepancy decisions;
- operator-specific lifecycle policy.

Use tables/code for:

- required field checks;
- enum checks;
- FK/existence checks;
- idempotency and duplicate detection;
- simple status transition guards;
- pagination/filter validation.

## 4.4 Frontend Read Composition

The riskiest performance screen is Customer 360 because it naturally wants ILM, SUB, BIL, TCK, WO, OSR, NOT, CVM, and SALES data.

Simplification:

- each panel loads independently;
- each panel has timeout and partial failure state;
- initial page loads only the customer header, active subscriptions, balance summary, open tickets, and recent timeline;
- older history, invoices, work orders, equipment, and campaign details load on tab/open;
- introduce a Backoffice BFF/read aggregator if direct gateway fan-out becomes slow;
- BFF/read aggregator must not write business state.

## 4.5 Reporting

REP-01 is not optional if the system needs management dashboards. Without REP-01, dashboards will overload operational APIs.

Simplification:

- v1 reports: daily revenue, payments, active subscribers, activations, tickets, work orders, stock summary, dunning status;
- near-real-time only for operational queues where needed;
- exports run async;
- dashboard APIs read REP-01 only;
- drill-down links may call source module read APIs for a specific customer/order/ticket, not for aggregate totals.

### 4.6 ASR/Ticketing

ASR-01/02/03/04 should not become four engines.

Simplification:

- TCK remains one ticket engine;
- ASR docs become ticket type definitions, rules, SLA profiles, escalation paths, and optional BPMN/user tasks;
- field intervention uses TCK -> WO API;
- customer-visible status remains mapped from TCK state.

### 4.7 Field Audit

FA-01/02/03 should not become three services.

Simplification:

- one field-audit capability;
- `audit_type = EQUIPMENT | NETWORK | CUSTOMER_KYC`;
- shared assignment/evidence/discrepancy tables where possible;
- type-specific rules and checklists only where needed;
- create WO only when a field visit is required.

### 4.8 CVM

Do not start with a full CVM platform.

Simplification:

- v1 CVM is rule-based queues and outcome tracking;
- use REP-01/BIL/SUB/TCK inputs;
- actions are notification, ticket/task, discount/campaign assignment, or follow-up;
- no ML scoring platform in v1.

### 4.9 Technology Migration

The latest correction is right: SUB-WF owns the business migration operation; FUL-10 owns the technical cross-technology fulfillment.

Simplification:

- do not create a separate WO migration engine unless needed;
- FUL-10 calls WO only for physical tasks;
- FUL-10 calls OSR/FUL-09 for equipment/service rebinding;
- PROV-INT handles provisioning dispatch/reconciliation;
- one correlation id ties the operation across modules.

## 5. Processes That Are OK As Designed

These should stay because simplifying them further would create hidden risk:

| Process | Why it should stay |
|---|---|
| SUB-LM vs SUB-WF split | Protects subscription master state from workflow complexity. |
| Billing separate from subscription | Prevents financial ledger pollution and clarifies money ownership. |
| WO separate from OSR | Field execution and equipment serial/stock ownership are different concerns. |
| TCK separate from WO | Ticket lifecycle and field execution lifecycle are different concerns. |
| Outbox/inbox | Required to make Kafka reliable with database commits. |
| REP-01 | Required to avoid live operational fan-out for reporting. |
| Payment gateway integration | Required for callback idempotency, settlement, and reconciliation. |
| Provisioning integration | Required to avoid ad hoc network adapter calls inside every workflow. |

# 6. Processes That Are Too Complex For First Release At Full Scope

These should be phased or reduced:

| Area | Risk if full scope is built immediately | Recommended v1 |
|---|---|---|
| FUL-02 all branches | Long onboarding process becomes hard to stabilize. | Build WIK happy path: capture, validate, KYC, payment, install, subscription, activation, cancellation. Add edge branches later. |
| Subscription MACD all flows | Too many flows before activation/revenue path is stable. | Build activation, suspend/resume, termination first; then upgrade/downgrade/relocation; migration last. |
| Campaigns/discount/CVM | Commercial logic can delay core BSS. | Basic discount assignment and campaign eligibility first; CVM simple queue later. |
| Procurement/inventory audit | Important but not needed to activate first customers if initial stock is seeded. | Seed stock + stock movements first; procurement/audit after stock basics. |
| Field audit | Useful but not core activation. | One audit engine, limited checklists, after WO/OSR are stable. |
| USSD | Needs telco integration and careful session handling. | Limited menus: balance, pay instruction/status, ticket creation/status. |
| Reporting portal | Large report catalog can grow endlessly. | Core operational/revenue dashboards only. |
| Mediation/rating | Offline rating is batch-heavy and can destabilize billing if rushed. | Implement after wallet/payment basics; batch, not online rating. |

# 7. Recommended MVP Waves

## Wave 1 - Revenue-Critical Core

Build only what is needed to sell, install, bill, support, and operate first customers:

- foundations;
- ILM customer/account/KYC basics;
- PLM/SIP/RLM catalogs needed for WIK products/HomePass;
- SUB-LM;
- SUB-WF framework and activation;
- BIL payment application, charging basics, invoice read/generation basics, wallet if prepaid requires it;
- PAY-GW only for the chosen first payment provider;
- FUL-02 happy path and FUL-03 activation;
- WO installation flow;
- OSR stock + equipment instance basics;
- EM-02 contractor/staff basics;
- NOT/ICN basics;
- TCK basic support ticket;
- Backoffice essential screens;

- contractor app essential install execution;
- REP-01 minimal operational dashboards.

### Wave 2 - Operational Hardening

- pause/resume/terminate;
- dunning and non-payment suspension;
- WO support/shifting;
- OSR-RMA/swap;
- provisioning reconciliation;
- tax invoice if mandatory for launch;
- customer self-care;
- reporting exports/reconciliation;
- API contract test and E2E automation expansion.

### Wave 3 - Commercial And Advanced Operations

- upgrade/downgrade/relocation/migration;
- campaigns, discounts, CVM;
- ASR specialized flows;
- procurement and inventory audit;
- field audits;
- USSD;
- mediation/rating;
- advanced franchise/sales attribution and commissions if needed.

## 8. Performance Assessment By Critical Journey

| Journey | Risk | Current design assessment | Required simplification |
|---|---|---|---|
| New sale to activation | High | Many modules participate, but FUL-02 as owner keeps it coherent. | Keep synchronous validation short; use workflow after acceptance; cache catalog/HomePass reads. |
| Customer 360 | High | Can become fan-out heavy. | Panelized reads, partial failure, optional BFF/read model. |
| Payment callback | High | Must be idempotent and fast. | PAY-GW validates/dedupes, BIL applies ledger, async notify/reporting. |
| Invoice generation | Medium-high | Batch pressure on billing DB. | Partition tables, run off peak, avoid per-customer synchronous fan-out. |
| Dunning suspension | Medium-high | Crosses BIL, SUB-WF, FUL, NOT. | Batch candidate selection, one operation per subscription, event-driven status. |
| WO finalization | Medium | Field app offline/evidence upload can be heavy. | Upload files to storage, WO stores metadata, OSR confirms equipment. |
| Reporting dashboard | High | Bad design would call every module. | REP-01 only for aggregates. |
| USSD balance/payment/ticket | Medium | Must be low-latency. | Minimal API calls, short sessions, no complex Customer 360 style screens. |
| Technology migration | High | Many moving parts. | Admin-only cohorts, FUL-10 orchestration, one correlation id, limited rollout. |

## 9. Implementation Rules To Add To Squad Kickoff

Every squad should follow these rules:

1. A DD is not a microservice by default.
2. A worker topic is not a pod by default.
3. A UI screen must name the command owner and read owners.
4. No module reads another module's DB.
5. No dashboard reads operational modules for aggregate totals.
6. Commands must be idempotent before side effects.
7. Events must be produced through outbox.
8. Cache misses must not break core command flows.
9. Camunda is for workflows, not CRUD.
10. Drools is for policy, not fixed validation.

# 10. Final Recommendation

Keep the design direction, but enforce an MVP runtime architecture:

- about 10-12 deployable groups, not 100+ services;
- 5-6 consolidated worker apps, not one worker per topic;
- REP-01 for reporting from day one;
- BFF/read aggregation only for high-fan-out backoffice screens;
- strict phasing of late DDs;
- monthly design review after implementation starts to retire unused complexity.

The system will fail operationally if every DD is implemented at full scope immediately. It will work if DDs are treated as clear ownership contracts and the first implementation is deliberately narrowed to the revenue-critical journeys.